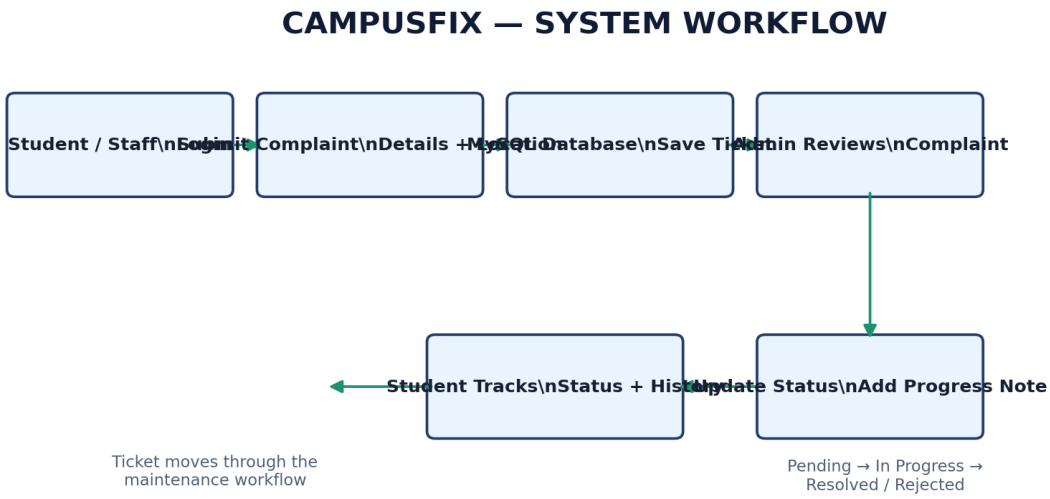
CAMPUSFIX

SMART CAMPUS COMPLAINT &
MAINTENANCE MANAGEMENT SYSTEM



COMPLETE PROJECT DOCUMENTATION

Report • Workflow diagram • Database design • Implementation guide • Test plan

Student Name: **Asia Waqar**	Supervisor: **Sara Fareed**
Department: **BS IT**	University: **Islamia University Bahawalpur**

CONTENTS

• Executive Summary
• Introduction and Background
• Problem Statement
• Aim and Objectives
• Scope and Stakeholders
• Proposed Solution and Features
• System Workflow
• Functional Requirements
• Non-Functional Requirements
• Technology Stack
• System Architecture
• Database Design
• Module Descriptions
• User Journey
• Installation Guide
• Test Plan
• Security and Privacy
• Limitations
• Future Enhancements
• Conclusion
• Setup Checklist
• Demonstration Script
• References

1. Executive Summary

CampusFix is a web-based Smart Campus Complaint & Maintenance Management System. Students and staff can report campus facility problems such as electrical faults, plumbing leaks, damaged furniture, cleaning needs, classroom/laboratory equipment faults, internet issues, and safety concerns. Every report becomes a structured ticket with a reference number, category, location, priority, description, status, and update history.

The prototype uses HTML5 and CSS3 for the interface, PHP for server-side processing, and MySQL/MariaDB for data storage. It includes registration and login, role-based access, complaint submission, status tracking, an admin control panel, progress notes, and timestamped status history. The project is intended for local academic demonstration; install and test it in your university's required environment before submission.

Project at a glance

Item	Description
Project type	Web-based information system / academic prototype
Users	Students, staff, campus administrator
Core process	Submit → Review → Update → Track → Resolve
Frontend	HTML5, CSS3
Backend	PHP
Database	MySQL / MariaDB
Environment	XAMPP, Apache, PHP, MySQL, phpMyAdmin

2. Introduction and Background

Campus facilities need regular maintenance to provide a safe and productive learning environment. Problems may occur in lecture rooms, laboratories, libraries, hostels, cafeterias, offices, and outdoor spaces. If reports are passed through verbal messages or separate chat groups, information can become scattered and users may not know whether work has started or finished. CampusFix provides a central record to submit and follow complaints.

3. Problem Statement

A manual or fragmented complaint process can make it difficult to record every issue, identify its location, prioritize urgent problems, and communicate progress. CampusFix stores complaint information in a structured database and presents different functions to ordinary users and administrators.

4. Aim and Objectives

Aim: Design and implement a web-based platform for transparent campus complaint reporting and maintenance tracking.
Objectives:
• Provide account registration and login.
• Record issue title, category, location, priority, and description.
• Generate a reference number for each complaint.
• Let users view their own status and update history.
• Give administrators a central view of all reports.
• Record status changes and progress notes.
• Use relational database design and basic security practices.

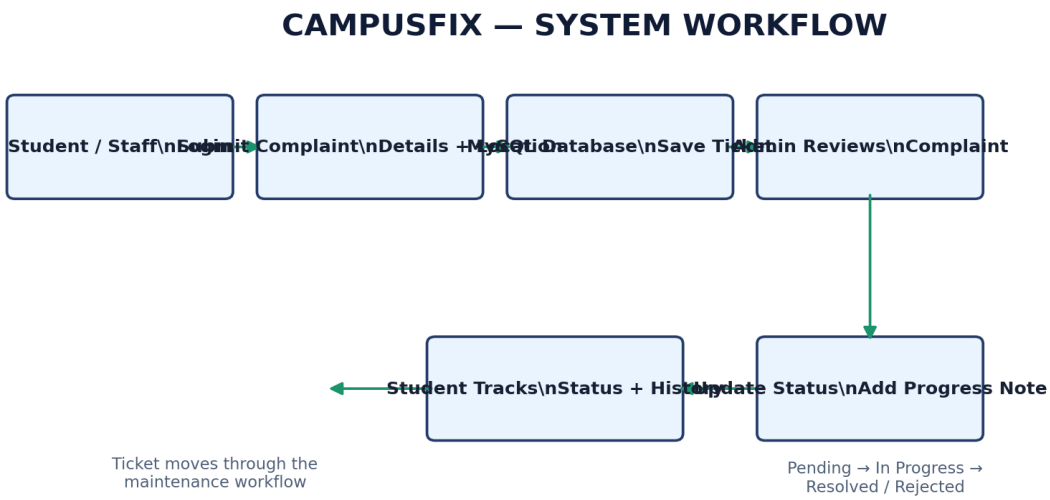
5. Scope and Stakeholders

Students/staff: submit complaints and monitor their own tickets.
Administrator: reviews reports, updates statuses, and records notes.
Maintenance team: represented through the admin workflow; direct technician accounts can be added later.
University management: may use summaries to understand maintenance demand.
Included: local web app, accounts, complaint entry, ticket list, status filters, admin updates, and history.
Not included yet: automatic SMS/email, photo uploads, maps, mobile app, hosting, or university single sign-on.

6. Proposed Solution and Features

Authentication: registration and login.
Complaint creation: structured form with validation.
Ticket tracking: unique reference ID and current status.
Admin dashboard: all complaints and status changes.
Update timeline: status changes recorded with account and timestamp.
Status filters: Pending, In Progress, Resolved, Rejected.
Responsive layout: pages adapt to desktop and mobile browser sizes.

7. System Workflow



Workflow explanation

1. User signs in or registers.
2. User enters the issue title, category, location, priority, and description.
3. PHP validates the form and stores the complaint in MySQL.
4. Administrator reviews the complaint.
5. Administrator updates status and may add a note.
6. User views current status and update history.
7. Ticket is marked Resolved when work is complete, or Rejected when not accepted.

8. Functional Requirements

ID	Requirement
FR-01	Users can register with name, email, and password.
FR-02	Private pages require an authenticated session.
FR-03	Non-admin users can submit complaints.
FR-04	The system stores complaint details and status.
FR-05	Users can view only their own complaints.
FR-06	Administrators can view all complaints.
FR-07	Administrators can update status and add notes.
FR-08	The system preserves a status-update history.
FR-09	Complaint lists can be filtered by status.
FR-10	Invalid forms show helpful feedback.

9. Non-Functional Requirements

Usability: clear navigation, labels, and feedback.
Security: authentication, role checks, password hashing, prepared statements, and output escaping.
Reliability: valid complaint records and related updates should be saved consistently.
Maintainability: shared configuration/layout and separate PHP pages.
Compatibility: modern browsers and PHP/MySQL local environment.
Performance: intended for small academic demonstrations; high-volume performance is not established.

10. Technology Stack

HTML5: page structure and forms.
CSS3: layout and responsive styling.
PHP: sessions, validation, access control, database queries.
MySQL/MariaDB: relational storage.
Apache/XAMPP: local server environment.
phpMyAdmin: database import and administration.

11. System Architecture

CampusFix uses a simple three-layer design: presentation layer (browser UI using HTML/CSS), application layer (PHP handles authentication, permissions, validation, and workflow), and data layer (MySQL stores users, complaints, and status history). Apache receives browser requests and runs PHP, which reads or writes database records and returns a page.

Browser UI → Apache/PHP application → MySQL database

12. Database Design

Table	Purpose / key fields
users	id, name, unique email, password_hash, role, created_at
complaints	id, user_id, title, category, location, priority, description, status, admin_note, timestamps
complaint_updates	id, complaint_id, user_id, status, note, created_at

Relationships

One user can submit many complaints. One complaint can have many update records. Each update identifies the account that made the change. Foreign keys link the records.

13. Module Descriptions

Home: introduces CampusFix and its workflow.
Register/Login: creates accounts and authenticated sessions.
Dashboard: shows counts and recent tickets.
New Complaint: validates and saves an issue.
Complaints: lists records and filters by status.
Details: shows issue information and status history.
Admin Panel: reviews all tickets and updates status/notes.

14. User Journey

A new user registers, logs in, opens the complaint form, enters the issue details, and submits the report. The system provides a reference ID. The administrator opens the ticket in the control panel and records progress. The user returns to the dashboard or complaint detail page to check the status.

15. Installation and Execution Guide

Requirements: XAMPP with Apache, PHP and MySQL/MariaDB; a modern browser.

Steps:
1. Extract the supplied ZIP.
2. Copy `campusfix` into XAMPP's `htdocs` folder.
3. Start Apache and MySQL.
4. Import `campusfix/database.sql` through phpMyAdmin.
5. Check credentials in `campusfix/config.php` (defaults: MySQL user `root`, blank password).
6. Open `http://localhost/campusfix/`.
7. Open `http://localhost/campusfix/setup_admin.php` once to create the demo admin; then delete that setup file.
8. Sign in with `admin@campusfix.local` / `Admin@123`.
9. Register a student account and submit a sample complaint.
10. Use the admin panel to update status, then verify the history from the student account.

Troubleshooting

Database error: ensure MySQL is running, import succeeded, and credentials match.
Page not found: check the folder is named `campusfix` and placed in `htdocs`.
Admin login fails: run setup script after importing database; remove it immediately after setup.
Port conflict: resolve the Apache port conflict and use the matching local URL.

16. Test Plan

ID	Test	Expected outcome
TC-01	Register valid user	Account created
TC-02	Duplicate email	Error feedback
TC-03	Login valid credentials	Dashboard opens
TC-04	Submit valid complaint	Ticket and reference created
TC-05	Required fields missing	Validation blocks save
TC-06	Student opens another user's ticket	Access denied
TC-07	Admin sets In Progress	Status/history updated
TC-08	Admin marks Resolved	Resolved shown to owner
TC-09	Filter by Pending	Only pending tickets shown
TC-10	Logout and revisit private page	Redirected to login

Testing note

Run these tests on the actual computer before presenting the project. Record the date, actual result, and Pass/Fail for each case. Do not state that tests passed until they have been executed.

17. Security and Privacy

The prototype uses password hashing, prepared SQL statements, output escaping, session ID regeneration at login, role checks, and CSRF tokens on POST forms. These controls help reduce common risks but do not make it production-ready. Before deployment, enable HTTPS, remove demo credentials and setup scripts, use least-privilege database credentials, add rate limiting and password reset, maintain backups, patch the server, and define data-retention rules.

18. Limitations

This academic version does not include automatic notifications, image uploads, technician assignment, SLA deadlines, report exports, single sign-on, or independent security/load testing. Operation depends on correct local server and database setup.

19. Future Enhancements

Possible additions include technician accounts, photo evidence, email/SMS alerts, QR codes for rooms, maps, maintenance deadlines, escalation rules, analytics, report export, university single sign-on, and a mobile progressive web app.

20. Conclusion

CampusFix demonstrates a structured workflow for campus maintenance reporting, from submission to administrator review and resolution tracking. It applies practical web development concepts including authentication, role-based access, relational database design, form validation, status transitions, and history tracking. It can be extended to meet additional university requirements after testing and review.

Appendix A. Setup Checklist

- ■ ZIP extracted
- ■ Folder copied to `htdocs`
- ■ Apache and MySQL started
- ■ SQL database imported
- ■ Credentials checked
- ■ Admin account created
- ■ Setup script deleted
- ■ Student account registered
- ■ Sample ticket submitted
- ■ Admin status update checked
- ■ Student history verified

Appendix B. Demonstration Script

1. Introduce the home page and project purpose.
2. Register a student and sign in.
3. Submit an issue such as a broken projector in a classroom.
4. Show the reference ID and Pending status.
5. Sign in as admin and open the control panel.
6. Change status to In Progress and add a note.
7. Mark the issue Resolved.
8. Return to the student account and show the updated history.

Appendix C. References

1. PHP Manual — https://www.php.net/manual/
2. MySQL Reference Manual — https://dev.mysql.com/doc/
3. MDN Web Docs — https://developer.mozilla.org/
4. Apache Friends / XAMPP — https://www.apachefriends.org/

Academic integrity note: Review the code, run the tests yourself, and follow your university's rules for attribution and AI-assisted work. Fill in your actual student name, supervisor, department, and university on the cover before submission.